TRNGs para generación de secuencias muy largas

por

Ciro Fabián Bermúdez Márquez

Tesis presentada en cumplimiento parcial de los requisitos para el grado de:

MSc. en Electrónica

en

Instituto Nacional de Astrofísica, Óptica y Electrónica (INAOE)

Julio, 2021

Santa María de Tonantzintla, Puebla

Asesores:

Dr. Esteban Tlelo Cuautle

Departamento de Electrónica INAOE

Dr. Cuauhtemoc Mancillas López

Departamento de Electrónica INAOE

©INAOE 2021

Todos los derechos reservados

El autor otorga a INAOE el permiso para reproducir y distribuir este documento.

Agradecimientos

Índice general

Agradecimientos I

Índice general III

Índice de figuras V

Índice de tablas VII

Lista de códigos IX

Resumen XI

1. Introducción 1

1.1. Fichas bibliográficas 1

1.1.1. Lista 1

1.2. Justificación 5

1.3. Objetivos 5

1.3.1. Objetivo general 5

1.3.2. Objetivos específicos 5

2. Fundamentos teóricos 7

3. Nombre del capitulo 11

3.1. Nombre de sección 11

4. Nombre del capitulo 13

4.1. Seccion del capitulo 13

5. Conclusiones 15

A. Códigos 17

Bibliografía 19

Índice de figuras

2.1. Arquitectura del núcleo ERO-TRNG. 7
2.2. Arquitectura del núcleo TERO-TRNG. 8

Índice de tablas

Lista de códigos

A.1. Nombre de abajo 17
A.2. Nombre de abajo 17

Resumen

Este trabajo de tesis presenta

Capítulo 1

Introducción

1.1. Fichas bibliográficas

1.1.1. Lista

1. 2021 - Luis Gerardo De La Fraga - Designing an authenticated Hash function with a 2D chaotic map. [1]

2. 2017 - Luis Gerardo De La Fraga - Hardware implementation of pseudo-random number generators based on chaotic maps. [2]

3. 2021 - Sundarapandian Vaidyanathan - A 5-D Multi-Stable Hyperchaotic Two-Disk Dynamo System With No Equilibrium Point: Circuit Design, FPGA Realization and Applications to TRNGs and Image Encryption. [3]

4. 2020 - N. Nalla Anandakumar - FPGA-Based True Random Number Generation Using Programmable Delays in Oscillator-Rings. [4]

5. 1999 - Benjamin Jun and Paul Kocher - The Intel Random Number Generator. [5]

6. 2021 - Xinyu Wang - High-Throughput Portable True Random Number Generator Based on Jitter-Latch Structure. [6]

7. 2021 - Adriaan Peetermans - Design and Analysis of Configurable Ring Oscillators for True Random Number Generation Based on Coherent Sampling. [7]

8. 2016 - Oto Petura - A Survey of AIS-20/31 Compliant TRNG Cores Suitable for FPGA Devices. [8]

9. 1993 - J.C. Sprott - Automatic generation of strange attractors. [9]

10. 2015 - Patrick Haddad - A Physical Approach for Stochastic Modeling of TERO-Based TRNG. [10]

11. 2010 - Mathieu Baudet - On the Security of Oscillator-Based Random Number Generators. [11]

12. 2010 - Michal Varchola - New High Entropy Element for FPGA Based True Random Number Generators. [12]

Los True Random Number Generators (TRNGs) se utilizan ampliamente en aplicaciones criptográficas para crear llaves aleatorias (random keys), parámetros en protocolos de desafió-respuesta, valores de relleno y máscaras. Para aplicaciones donde la seguridad no es un aspecto crítico el método de Monte-Carlo, simulaciones de computadora, modelado, aplicaciones de análisis numérico y algoritmos probabilísticos. Además se pueden utilizar en el día a día en sorteos de lotería, juegos de computadora, juegos de azar. [3] [4] [10].

Los generadores de números aleatorios se dividen en dos categorías principales, los generadores de números aleatorios deterministic random number generators (DRNG) y los true random number generators (TRNG), los cuales pueden ser físicos (P-TRNG) o no físicos (NP-TRNG). Mientas que los generadores deterministicos se basan en procesos algorítmicos y, por tanto, no son verdaderamente aleatorios, los TRNGs explotan un proceso impredecible, como los fenómenos analógicos de los dispositivos electrónicos, para producir una secuencia binaria aleatoria o una secuencia de números aleatorios. La imprevisibilidad de los DRNG se garantiza computacionalmente y la de los TRNG se garantiza físicamente. Por tanto, es necesario conocer bien el proceso físico subyacente en los TRNG que garantiza su aleatoriedad y, por tanto, su imprevisibilidad.

La calidad estadísticas de los TRNGs se evalúan normalmente utilizando conjuntos de pruebas estadísticas como las pruebas NIST o DIEHARD, además de realizar análisis de correlación y entropía de acuerdo al tipo de información o aplicación que se este desarrollando.

Para construir y probar TRNGs usualmente se utilizan Field-Programmable Gate Arrays (FPGAs) debido a su flexibilidad, alto rendimiento y bajo costo en comparación con otras alternativas. Sin embargo la implementación de TRNGs en FPGAs es un reto debido a los limitados recursos y técnicas específicas de TRNG disponibles para el diseñador. Debido a la disponibilidad de recursos digitales en las FPGAs, estos TRNGs se basan normalmente en la imprevisibilidad de los elementos de memoria metaestable o en el jitter de tiempo presente en los osciladores de funcionamiento libre (free-running oscillators). Muchos TRNG se han publicado en los últimos años sin embargo muchos sufren de la dependencia de fuentes de entropía externas, convertidores analógicos digitales o de recursos propios de una familia de FPGAs lo que los hace poco prácticos para su uso continuo y portabilidad. Se debe hacer entonces una distinción entre los núcleos TRNGs que sean candidato dadas ciertas características que favorezcan su desarrollo y mejora continua. [8]

Estos núcleos TRNG deben cumplir varios requisitos: 1) su diseño debe ser sencillo y comprensible, la fuente de aleatoriedad debe estar claramente definida; 2) el proceso aleatorio subyacente debe ser estacionario y el modelo estocástico debe ser factible; 3) la señal binaria bruta (raw) debe estar disponible para posteriores pruebas fuera de línea y en línea. Además, es útil que la fuente de aleatoriedad (por ejemplo, la clock

jitter procedente del ruido térmico) sea cuantificable: su medición dentro del dispositivo puede servir de base para realizar pruebas estadísticas integradas rápidas y eficaces.

Los núcleos TRNG que utilizan circuitos oscilantes son los siguientes:

1. Ring Oscillator Based Elementary TRNG (ERO-TRNG)
2. Ring Oscillator Coherent Sampling Based TRNG (COSO-TRNG)
3. Multi-Ring Oscillator Based TRNG (MURO-TRNG)
4. Coherent Sampling Based TRNG Using PLLs (PLL-TRNG)
5. Transition Effect Ring Oscilaltor Based TRNG (TERO-TRNG)
6. Self Timed Ring Based TRNG (STR)

En todos los artículos que se han leído se tiene la misma visión que cuando se trata con TRNGs es importante tener una **alta entropía** de los números generados, **alto rendimiento** (throughput), **portabilidad**, y pasar las pruebas estándar.

En muchas mayoría de los casos es necesario realizar un posprocesamiento para aumentar o disminuir alguna de las características del TRNG.

No existe una señal digital ideal en un circuito real. Debido al ruido térmico, los flancos de subida y bajada de la forma de onda de muestreo a veces se adelantan o se retrasan, lo que se llamado jitter.

1.2. Justificación

1.3. Objetivos

1.3.1. Objetivo general

Objetivo

1.3.2. Objetivos específicos

1. Objetivo

Capítulo 2

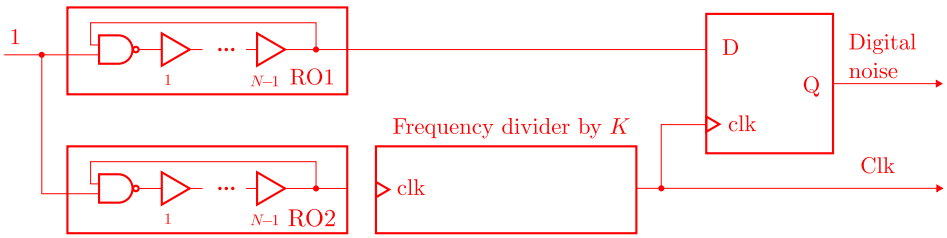
Fundamentos teóricos

Cuando evaluamos diferentes implementaciones en FPGAs de TRNGs hay que tener en cuenta el área, el bit rate de salida, los requerimientos de potencia y la calidad estadística de la salida.

1. Ring Oscillator Cased Elementary TRNG (ERO-TRNG)

   El ring oscillator based elementary TRNG (ERO-TRNG) fue propuesto y modelado en [11]. El diagrama de bloques del ERO-TRNG implementado en FPGAs se representa en la Figura 2.1.

   Figura 2.1: Arquitectura del núcleo ERO-TRNG.



   El generador utiliza dos osciladores de anillo idénticos (RO1 y RO2) como fuentes de aleatoriedad. La salida de uno de los anillos (una señal de reloj con jittery) se muestrea en un flip-flop D tras un periodo de acumulación suficientemente largo derivado de la segunda señal de reloj mediante un divisor de frecuencia por K (un contador síncrono de 17 bits). El número de elementos del anillo (N) se elije para obtener aproximadamente la misma frecuencia de reloj (300 MHz). El periodo de muestreo (y, por tanto, el parámetro K) se seleccionó en función de la frecuencia de reloj y del tamaño del jitter.

   Los dos osciladores de anillo se colocan y conectan manualmente para garantizar la repetibilidad del diseño. Aunque los dos anillos se colocaron muy cerca, aparentemente no se bloquearon.

El área del TRNG es relativamente pequeña a costa de una baja tasa de bits de salida.

El ERO-TRNG tiene una tasa de bits de salida muy pequeña, pero también un potencial de seguridad muy alto: para garantizar una tasa de entropía suficiente para el factor de división K dado, basta con que la prueba incrustada compruebe que los anillos están oscilando y que no están bloqueados.

Una mayor frecuencia de reloj aumenta la tasa de bits de salida a costa de un mayor consumo de energía

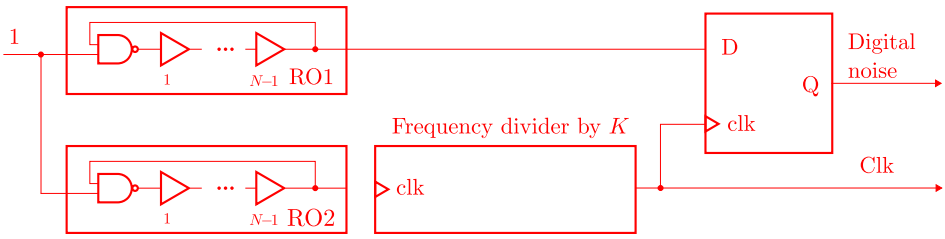
Resumen de características

- Recursos: dos ring oscillators (RO1 y RO2) idénticos como fuente de aleatoriedad, un flip-flop D, un divisor de frecuencia por K.

2. Ring Oscillator Coherent Sampling Based TRNG (COSO-TRNG)
3. Multi-Ring Oscillator Based TRNG (MURO-TRNG)
4. Coherent Sampling Based TRNG Using PLLs (PLL-TRNG)
5. Transition Effect Ring Oscilaltor Based TRNG (TERO-TRNG)

El TRNG basado en el oscilador de anillo de efecto de transición (TERO-TRNG) fue propuesto en [12] y su modelo estocástico en [10]. El diagrama de bloques del núcleo TERO-TRNG implementado en dispositivos FPGA se representa en la Figura 2.2.

Figura 2.2: Arquitectura del núcleo TERO-TRNG.



El oscilador de anillo de efecto de transición (TERO) es un oscilador de anillo de múltiples eventos con colisiones construido como un bucle de puertas lógicas. El bucle contiene un número par de puertas inversoras y un número cualquiera de puertas no inversoras. Debido al número par de compuertas inversoras, el oscilador tiene que reiniciarse con regularidad: los dos eventos creados después de cada reinicio circulan dentro del bucle hasta que se produce una colisión, durante la cual el flanco que se mueve más rápido alcanza al más lento.

La diferencia en la velocidad de los eventos circulantes está causada por las diferencias en los retrasos entre los inversores en las ramas del bucle y por los

fenómenos analógicos en los inversores y los buffers. Los eventos circulantes crean oscilaciones temporales que desaparecen tras la colisión.

El corazón del TERO-TRNG es la celda TERO, a la que sigue un contador (en la Figura 2.2, el contador está representado por un flip-flop T) y un registro de datos de salida.

El área del núcleo del TRNG es muy pequeña, mientras que la tasa de bits de salida puede ser relativamente alta. La arquitectura del TRNG es sencilla, pero para conseguir una tasa de entropía suficiente, las dos ramas de la célula TERO deben estar desequilibradas de tal forma que el número de periodos de oscilación M cumpla la siguiente condición:

Resumen de caracteristicas

6. Self Timed Ring Based TRNG (STR)

$$\begin{aligned} x_{n+1} &= a_1 + a_2 x_n + a_3 x_n^2 + a_4 x_n y_n + a_5 y_n + a_6 y_n^2 \\ y_{n+1} &= a_7 + a_8 x_n + a_9 x_n^2 + a_{10} x_n y_n + a_{11} y_n + a_{12} y_n^2 \end{aligned} \tag{2.1}$$

donde los parámetros $\{a_1, a_2, \ldots a_{12}\}$ pueden tomar diferentes valores, sin embargo para este trabajo se usaron los siguientes

Capítulo 3

Nombre del capitulo

3.1. Nombre de sección

Capítulo 4

Nombre del capitulo

4.1. Seccion del capitulo

Capítulo 5

Conclusiones

Apéndice A

Códigos

```
clear; close all; clc;
iter = 100000;
n = 0:1:iter;
a = [-0.6, -0.1, 1.1, 0.2, -0.8, 0.6, -0.7, 0.7, 0.7, 0.3, 0.6, 0.9];        %M1
x0 = 0.1; y0 = 0.2;
xn = zeros(1,iter+1); yn = zeros(1,iter+1);
xn(1) = x0; yn(1) = y0;
for i = 2:iter+1
    xn(i) = a(1) + a(2)*xn(i-1) + a(3)*xn(i-1)^(2) + a(4)*xn(i-1)*yn(i-1) + a(5)*yn(i-1) + a(6)*yn(i-1)^(2);
    yn(i) = a(7) + a(8)*xn(i-1) + a(9)*xn(i-1)^(2) + a(10)*xn(i-1)*yn(i-1) + a(11)*yn(i-1) + a(12)*yn(i-1)^(2);
end
plot(xn,yn,'.k','MarkerSize',0.5); grid on; xlabel("xn"); ylabel("yn"); axis equal;
```

Código A.1: Nombre de abajo

```
library ieee;
use ieee.std_logic_1164.all;

entity mux is
  generic( n : integer := 64);  -- Tamanio de palabra
  port(
        X0  : in std_logic_vector(n-1 downto 0);
        Xn_1: in std_logic_vector(n-1 downto 0);
        SEL : in std_logic;
        Xn  : out std_logic_vector(n-1 downto 0)
  );
end;

architecture arch of mux is
begin
  Xn <= X0 when SEL = '0' else Xn_1;
end arch;
```

Código A.2: Nombre de abajo

Bibliografía

[1] L. G. D. la Fraga, C. Mancillas-López, and E. Tlelo-Cuautle, "Designing an authenticated hash function with a 2d chaotic map," *Nonlinear Dynamics*, vol. 104, pp. 4569–4580, may 2021.

[2] L. G. de la Fraga, E. Torres-Pérez, E. Tlelo-Cuautle, and C. Mancillas-López, "Hardware implementation of pseudo-random number generators based on chaotic maps," *Nonlinear Dynamics*, vol. 90, pp. 1661–1670, aug 2017.

[3] S. Vaidyanathan, A. Sambas, B. Abd-El-Atty, A. A. A. El-Latif, E. Tlelo-Cuautle, O. Guillen-Fernandez, M. Mamat, M. A. Mohamed, M. Alcin, M. Tuna, I. Pehlivan, I. Koyuncu, and M. A. H. Ibrahim, "A 5-d multi-stable hyperchaotic two-disk dynamo system with no equilibrium point: Circuit design, FPGA realization and applications to TRNGs and image encryption," *IEEE Access*, vol. 9, pp. 81352–81369, 2021.

[4] N. N. Anandakumar, S. K. Sanadhya, and M. S. Hashmi, "FPGA-based true random number generation using programmable delays in oscillator-rings," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, pp. 570–574, mar 2020.

[5] B. Jun and P. Kocher, "The intel random number generator," *Cryptography Res. Inc., San Francisco, CA, USA*, 1999.

[6] X. Wang, H. Liang, Y. Wang, L. Yao, Y. Guo, M. Yi, Z. Huang, H. Qi, and Y. Lu, "High-throughput portable true random number generator based on jitter-latch structure," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, pp. 741–750, feb 2021.

[7] A. Peetermans, V. Rožić, and I. Verbauwhede, "Design and analysis of configurable ring oscillators for true random number generation based on coherent sampling," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 14, pp. 1–20, jun 2021.

[8] O. Petura, U. Mureddu, N. Bochard, V. Fischer, and L. Bossuet, "A survey of AIS-20/31 compliant TRNG cores suitable for FPGA devices," in *2016 26th International Conference on Field Programmable Logic and Applications (FPL)*, IEEE, aug 2016.

[9] J. Sprott, "Automatic generation of strange attractors," *Computers & Graphics*, vol. 17, pp. 325–332, may 1993.

[10] P. Haddad, V. Fischer, F. Bernard, and J. Nicolai, "A physical approach for stochastic modeling of TERO-based TRNG," in *Lecture Notes in Computer Science*, pp. 357–372, Springer Berlin Heidelberg, 2015.

[11] M. Baudet, D. Lubicz, J. Micolod, and A. Tassiaux, "On the security of oscillator-based random number generators," vol. 24, pp. 398–425, oct 2010.

[12] M. Varchola and M. Drutarovsky, "New high entropy element for FPGA based true random number generators," pp. 351–365, Springer Berlin Heidelberg, 2010.